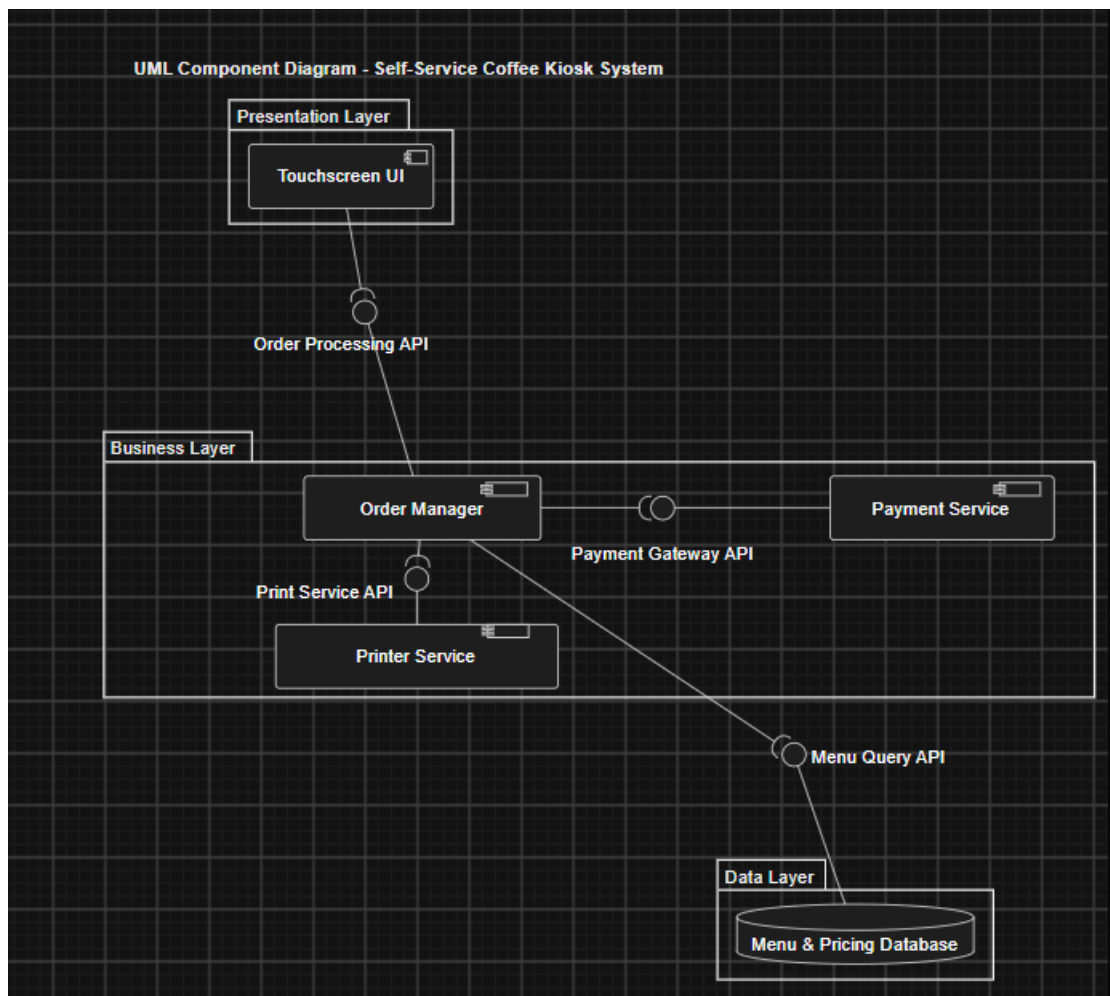


Lab 3: Component Modelling and Architectural Pattern Selection

PROBLEM STATEMENT #16 | Healthcare & Telemedicine Remote Patient Vitals Alert & Monitoring App

Student Name	Tushar N
SRN	PES1UG24CS504
Problem Statement	#16 – Healthcare & Telemedicine

- **Component Diagram**



- **Architecture Selection and Justification**

1. Architecture Selection

We chose Layered Architecture (3-Tier Layered Style) for the Self-Service Coffee Kiosk System.

The Self-Service Coffee Kiosk operates as a self-contained, high-reliability standalone terminal with physical hardware peripherals (Touchscreen, Receipt Printer, Credit Card Terminal). A Layered Architecture organizes the application logically into horizontal layers—Presentation Layer (Touchscreen UI), Business Layer (Order Manager, Payment Service, Printer Service), and Data Layer (Menu & Pricing Database)—ensuring a strict separation of concerns across the system.

2. Scenario-Related Justifications

- Reason 1 (Hardware & I/O Isolation for Kiosk Peripherals):

The kiosk must directly interface with physical hardware, including touchscreens and receipt printer hardware. A layered architecture cleanly abstracts device drivers into the Presentation/Hardware layer. If the receipt printer hardware or touchscreen driver is changed or upgraded, only the Presentation Layer needs to be modified, leaving core order rules (coffee types, pricing, menu logic) intact and unaffected.

- Reason 2 (Standalone Reliability & Low Operational Complexity):

Unlike a Microservices architecture—which introduces high network latency, service mesh overhead, and complex inter-service communication unsuitable for a single physical terminal—a monolithic Layered Architecture runs efficiently on local hardware. It eliminates external network dependencies for order assembly, local menu lookups, and receipt formatting, providing maximum uptime in a busy café environment.

3. Security Advantage & Performance Benefit

- Security Advantage (PCI-DSS Isolation & Strict Domain Enclosure):

By encapsulating credit card processing inside a dedicated Payment Service Component in the Business Layer behind strict interface boundaries, sensitive financial data is completely isolated from UI handlers and menu components. Communication occurs over encrypted, restricted interface calls (Payment Gateway API), minimizing PCI-DSS audit scope and preventing UI-level scripts from accessing credit card tokens.

- Security / Performance Benefit (Zero Network Latency for Core Workflows):

In-memory function calls between adjacent layers (UI → Order Manager → Database) deliver sub-millisecond response times for menu browsing, drink size selection (Small, Large), and real-time total price calculation. This guarantees instant touchscreen responsiveness during peak café hours without risking delays from external network calls.